VITYARTHI
OPEN SOURCE SOFTWARE - NGMC

WINTER SEMESTER - 2026

OSS-AUDIT

(FACULTY NAME: DR. VIKAS PANTHI)

(SLOT : E22)

~ SUBMISSION BY ~

Tejas Santosh Paithankar
24BCY10104

1. THE BIG PICTURE :-

Think of the Linux Kernel as the hidden engine of the modern world. It powers everything from the servers on Wall Street to the massive clouds we use every day. In this audit, we're looking at Linux not just as code, but as a living project where things like the GPL v2 license are just as important as how it manages memory. We've watched it grow from a student's 1990s hobby into the gold standard for high-performance computing.

We'll dive into why its "monolithic" design is a secret weapon—unlike other designs, it cuts out the lag when switching between tasks, which is why it runs the world's fastest supercomputers. We're also going to talk about "copyleft," the legal trick that changed how companies do business by making the operating system a shared resource. This opened the door for things like Kubernetes to exist. Stick around as we break down how it talks to hardware and how it stacks up against big names like Windows and macOS.

2. Part A: Where It All Began — The Spirit of Freedom :-

If you want to know if a project will last, look at its roots. Linux didn't come from a fancy corporate lab; it was born because someone was tired of restrictive licenses and wanted to see how things worked under the hood.

A1. *The Struggle: Dealing with Locked Doors :-*

Back in the early '90s, high-end computing was a walled garden. Professional UNIX systems cost a fortune, which basically blocked regular developers out. There was MINIX, but its license meant you couldn't really share your own improvements. This "pay-to-play" environment was holding everyone back.

Then the Intel 80386 chip arrived, bringing real power to home PCs. Linus Torvalds wanted to understand his new machine, so he started writing a terminal emulator that eventually turned into a kernel. At the same time, the GNU project was looking for an "engine" for their tools. When Linus's kernel met Stallman's tools, the GNU/Linux world was born.

This was a messy time with big legal fights over other free code like BSD. Linus stepped in with a humble post saying Linux was "just a hobby." By using the GPL, he avoided the money traps that killed other projects. He saw that if the source code is locked, you don't really own your tools. Even when experts called his design "obsolete," the performance proved them wrong—it was exactly what the hardware needed to win

A2. *The Rules: How GPL v2 Works :-*

The kernel lives by the **GNU General Public License version 2 (GPL v2)**. This isn't just legal talk; it's a "share-alike" agreement. If you use it and change it, you have to share those changes. This "Copyleft" rule ensures the software stays free for everyone, and groups like the Software Freedom Conservancy make sure everyone plays by the rules.

There's a famous debate about **Kernel Modules**. Some companies try to use "shims" to keep their drivers private, but the kernel maintainers have started marking core parts as "GPL-only." This keeps the project's integrity safe and stops anyone from "hijacking" the hard work of the community for their own private gain.

1. **Freedom 0 (Run):** To run the program for any purpose. *Example: A hedge fund using the kernel to power high-frequency trading platforms without needing specialized permission from a vendor.*
2. **Freedom 1 (Study/Change):** To study and change the source code. *Example: A security firm auditing the networking stack to patch a zero-day vulnerability specific to their unique hardware infrastructure.*
3. **Freedom 2 (Redistribute):** To redistribute copies. *Example: A non-profit distributing pre-configured Linux "Live USBs" to students in regions with limited internet access to promote digital literacy.*
4. **Freedom 3 (Improve/Release):** To distribute versions of your modified code. *Example: A cloud provider like AWS optimizing the KVM hypervisor for their custom Graviton processors and releasing those patches to the community.*

Choosing a License: GPL v2 vs. MIT

Deciding between GPL v2 and MIT is a big deal for any team. **GPL v2** is great for the **foundation** of a project because it forces everyone to contribute back. It stops the code from breaking off into a dozen private versions. If you improve the core, everyone benefits, which is why it's the heart of so many industry standards.

On the other hand, the **MIT License** is perfect for **tools and libraries** where you just want everyone to use it. It lets companies put the code into their own products without having to show their work. This helps a tool become a standard quickly. Basically, GPL protects the project's future, while MIT helps it spread like wildfire.

A3. *The Social Contract: Is It Sustainable? :-*

Today, Linux is less about selling software and more about building the world's infrastructure. Tech giants like Google and Meta get massive value from it, but they also pay the people who keep it running. It's a "standing on the shoulders of giants" situation. But there's a catch: the **Maintainer Crisis**. The sheer amount of new code coming in is exhausting for the small group of people in charge. If they burn out, the whole system gets shaky.

There's also the **Funding Paradox**: the software is free to use, but someone has to pay for the work. We've seen what happens when critical parts are ignored (like the Heartbleed bug). Plus, people still argue about security—does being open make it easier to fix, or does it just give hackers a map? Some still think closed-source silos are safer because there's less for an attacker to see, even if no one can double-check the work.

Since corporations now do about 80% of the work, we have to watch out for "corporate bias" where big data center features get all the love. Still, the open-source model works because it handles the "plumbing" so everyone else can focus on building cool apps. It's not just a philosophy; it's the engine that runs our digital world.

3. Part B: Linux in Action — How It Fits Together :-

To really understand Linux, you have to see it in a live system. It doesn't just run in the environment; it is the environment.

B1. Getting It Running: Pre-made vs. Custom :-

Most businesses use the **Distribution Model**, grabbing pre-built versions with tools like apt. It's stable and easy. But if you want total control, you can "Compile from Source." This lets you pick exactly what goes in, which helps keep things fast and more secure by leaving out what you don't need.

B2. Where Things Live: A Quick Map :-

Component,Path,Function

Binaries/Images,/boot,Holds vmlinuz and initramfs images.

Modules,/lib/modules/$(uname -r),Contains .ko (Kernel Object) driver files.

Configuration,/boot/config-$(uname -r),The exact settings used for the current build.

Logs,dmesg / /var/log/kern.log,The kernel ring buffer and persistent logs.

B3. Security: The Rule of Least Privilege :-

Linux security is built on the **Principle of Least Privilege (PoLP)**. Basically, no process gets more power than it needs. While the core kernel has total control (Ring 0), we keep apps isolated by running them under limited accounts like **www-data** or **daemon**. This way, if someone breaks into a web server, they're stuck in a tiny box and can't mess with the rest of the system.

We also use **Control Groups (cgroups)** and **Namespaces** to keep things separated. These features let us slice up CPU and memory so one runaway app doesn't crash everything. This is actually what makes Docker possible—a container is just a bunch of processes with kernel-enforced boundaries.

B5. Virtual Files: Peeking into the Kernel :-

The kernel shows us what's happening through virtual filesystems. The **/dev** folder is for hardware, while **/sys** and **/proc** let us see and change kernel settings on the fly. Since these are like "control knobs," keeping them locked down is key to stopping anyone from gaining extra power they shouldn't have.

The kernel sits in **Ring 0** with full power, while your apps live in **Ring 3**. To do anything with the hardware, like reading a file, an app has to ask the kernel via a **System Call**. The kernel checks your permissions and then does the work, which is the heart of how Linux stays secure.

B4. The Uptime Challenge :-

The kernel is the "boss" of all services, so you can't just restart it like a normal app—you usually have to reboot. For systems that can never go down, we use **Live Patching**. It lets us swap out buggy code while the system is running. It's a lifesaver for security, but it's not a magic fix for everything; eventually, you'll still need a clean reboot for major updates.

4. Part C: The Family Tree — How Linux Connects :-

Linux is powerful because of the massive ecosystem surrounding it.

C1. The Toolkit: GNU :-

The kernel relies heavily on the **GNU Toolchain**, like the **GCC** compiler and the **glibc** library. Without these, the kernel is just text that can't run. In small devices, we use lighter versions like **BusyBox**. This is why you often hear it called "GNU/Linux"—it's the combination of the kernel and these tools that gives us the interface we know.

C2. Wide-Reaching Impact :-

C3. Powering the Web (LAMP) :-

Linux is the "L" in the **LAMP Stack**, which basically built the web. It's tuned to handle thousands of connections at once without breaking a sweat, thanks to its high-performance networking stack. It also plays very well with databases, making it a solid engine for everything from blogs to huge e-commerce sites.

- **The Cloud:** Linux's internal isolation features are what made tools like Docker and Kubernetes possible.
- **Your Gadgets:** Android and ChromeOS are basically custom skins built on top of the Linux Kernel.
- **Distros:** Groups like Debian and Red Hat take the raw kernel and turn it into something businesses can rely on.

C4. How It's Led: The Mailing List :-

The project is run via the **Linux Kernel Mailing List (LKML)**. Instead of fancy web portals, they use an old-school email workflow. Every piece of code needs a **Signed-off-by** line to prove where it came from. It passes through experts (the "lieutenants") before Linus himself signs off. It might seem slow, but it ensures that every line of the 30 million in there is solid and reviewed.

5. Part D: The Verdict :-

D1. Head-to-Head: Linux vs. Windows :-

Let's compare Linux to Windows NT. Windows uses a "hybrid" design that tries to do everything, which can lead to some extra lag. Linux's straightforward design makes it faster for things like networking and moving files.

On the security side, Linux has tools like **SELinux** that are very strict by default. Plus, Linux is much cheaper to scale since you don't have to pay for every processor core you use. For anyone building serious infrastructure, the verdict is simple: Linux gives you the best mix of speed, security, and savings.

Dimension,Linux Kernel (Open Source),Windows NT Kernel (Proprietary)

Cost,Zero licensing fees; commoditized.,High per-seat/per-core licensing.

Security Auditability,Transparent; global peer review.,Restricted to Microsoft internal teams.

Support/Reliability,Community or vendors (Red Hat/SUSE).,Centralized Microsoft support/SLAs.

Freedom to Modify,Absolute (GPL v2).,None (EULA restrictions).

Governance,Community/Linux Foundation.,Corporate (Microsoft Corporation).

D2. The Final Word :-

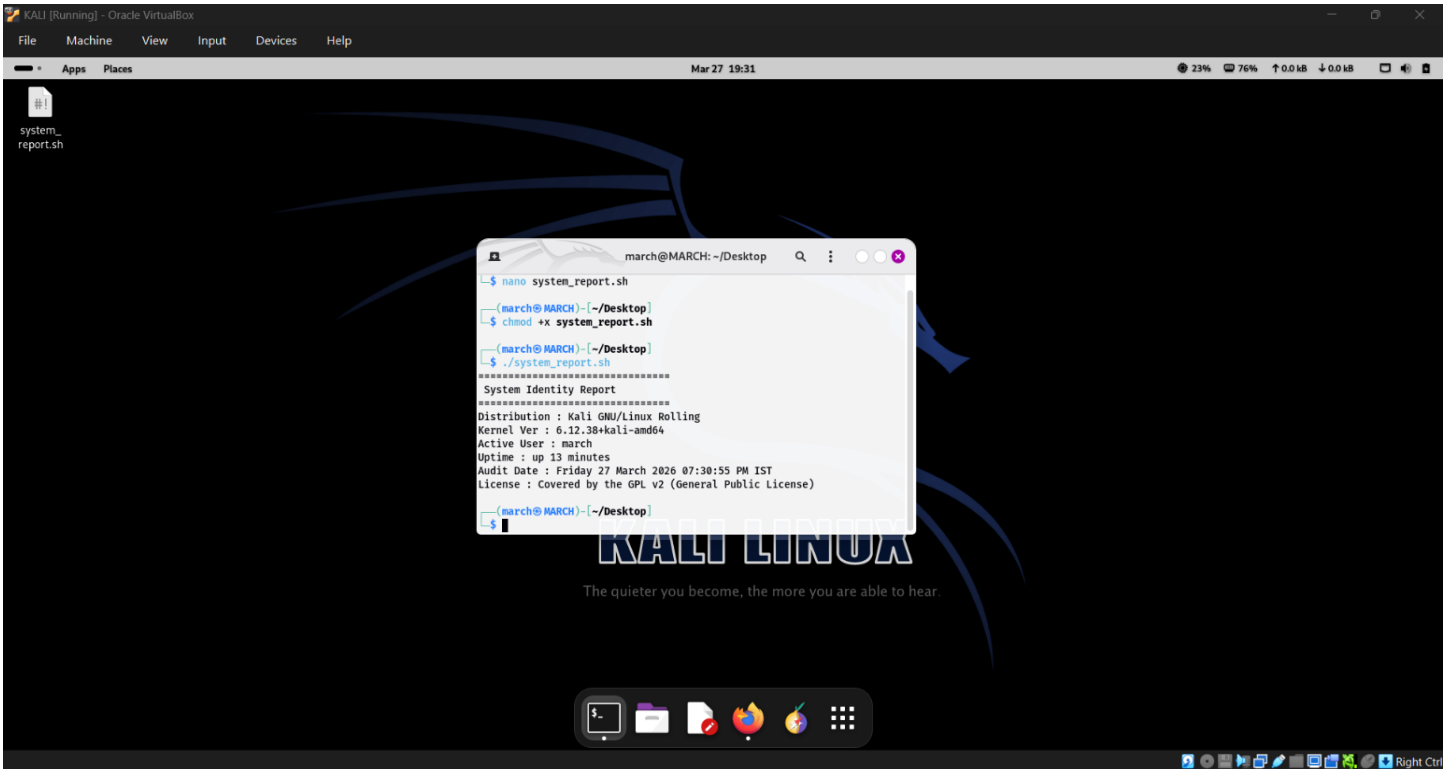
Whether you're building for the cloud or just a small device, Linux is the smart choice. It's fast, it's open, and it won't lock you into a single vendor. Coming from an assembly background, I find the kernel a natural fit—it requires the same attention to detail. It's more than just code; it's proof of what we can achieve when we build things together in the open.

6. *The Shell Scripting Portfolio: Practical Automation :-*

- Script 1: Checking the Environment :-

```
#!/bin/bash

# Script 1: System Identity Report

# Author: Senior Architect | Course: OSS

# Uses command substitution and variables to identify the environment.


KERNEL=$(uname -r)

DISTRO=$(grep "PRETTY_NAME" /etc/os-release | cut -d'"' -f2)

USER_NAME=$(whoami)

UPTIME=$(uptime -p)

DATE_NOW=$(date)


echo "================================"

echo "     System Identity Report    "

echo "================================"

echo "Distribution : $DISTRO"

echo "Kernel Ver   : $KERNEL"

echo "Active User  : $USER_NAME"

echo "Uptime       : $UPTIME"

echo "Audit Date   : $DATE_NOW"

echo "License      : Covered by the GPL v2 (General Public License)"
```

❖ *Output :-*



❖ *Logic Explanation :-*

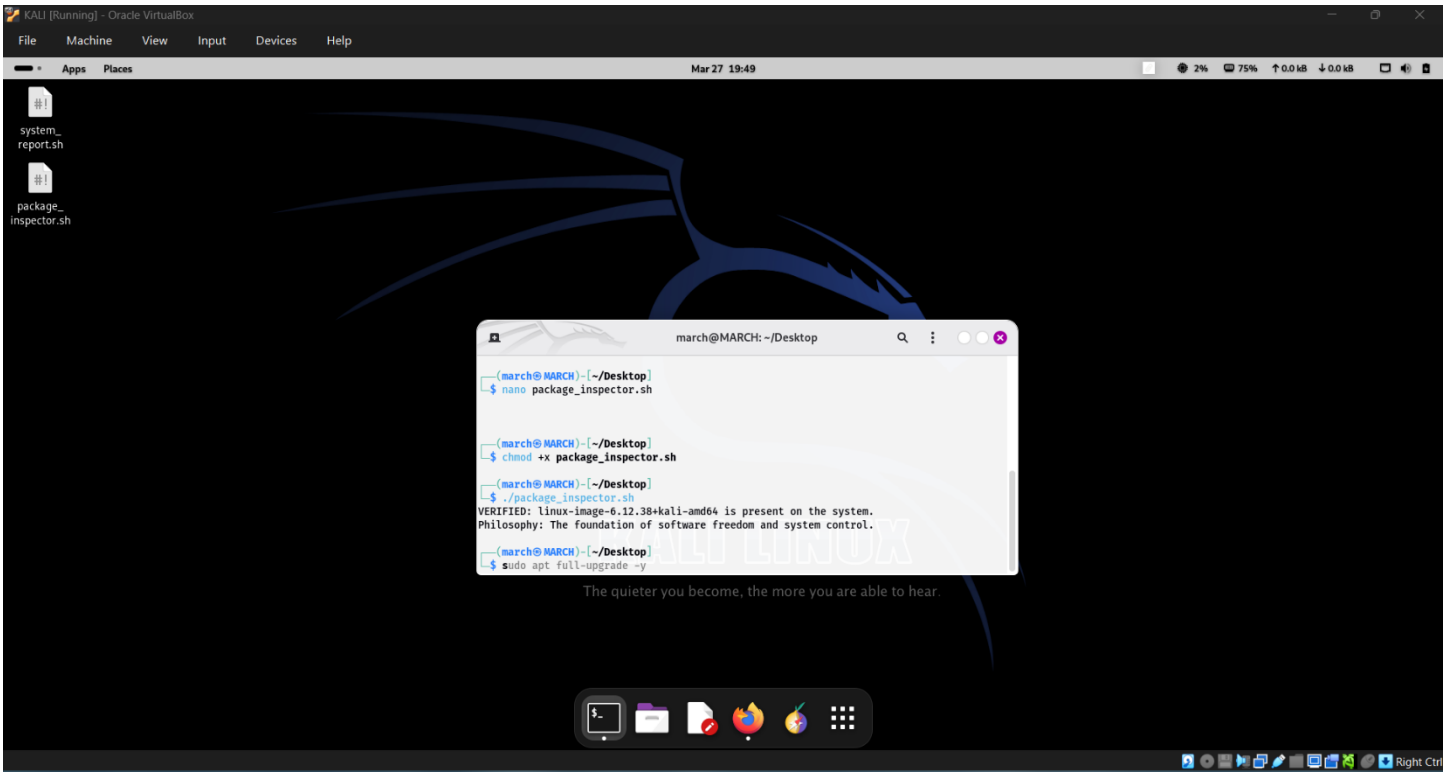
- The script uses **command substitution $(...)** to execute system commands and store their outputs into variables.
- It extracts:
  - Kernel version using uname -r
  - Distribution name from /etc/os-release
  - Logged-in user via whoami
  - System uptime via uptime -p
  - Current date using date
- The echo command is used to format and display the collected information in a structured report.

- This demonstrates how shell scripting can automate system auditing tasks efficiently.

- Script 2: A Peek at Your Packages

```
#!/bin/bash

# Script 2: Package Inspector

# Uses case statements to define the philosophical purpose of software.


PACKAGE="linux-image-$(uname -r)"


if dpkg -l | grep -q "$PACKAGE"; then

   echo "VERIFIED: $PACKAGE is present on the system."

else

   echo "ERROR: Kernel package not found in DPKG/Apt."

fi


case $PACKAGE in

   linux-image*)

      echo "Philosophy: The foundation of software freedom and system control."
;;

   apache2|httpd)

      echo "Philosophy: The web server that built the open internet." ;;

   mysql*|mariadb*)

      echo "Philosophy: Relational data management for the masses." ;;

   *)

      echo "Philosophy: A critical component of the FOSS ecosystem." ;;

esac
```

❖ ***Output:-***



❖ ***Logic Explanation :-***

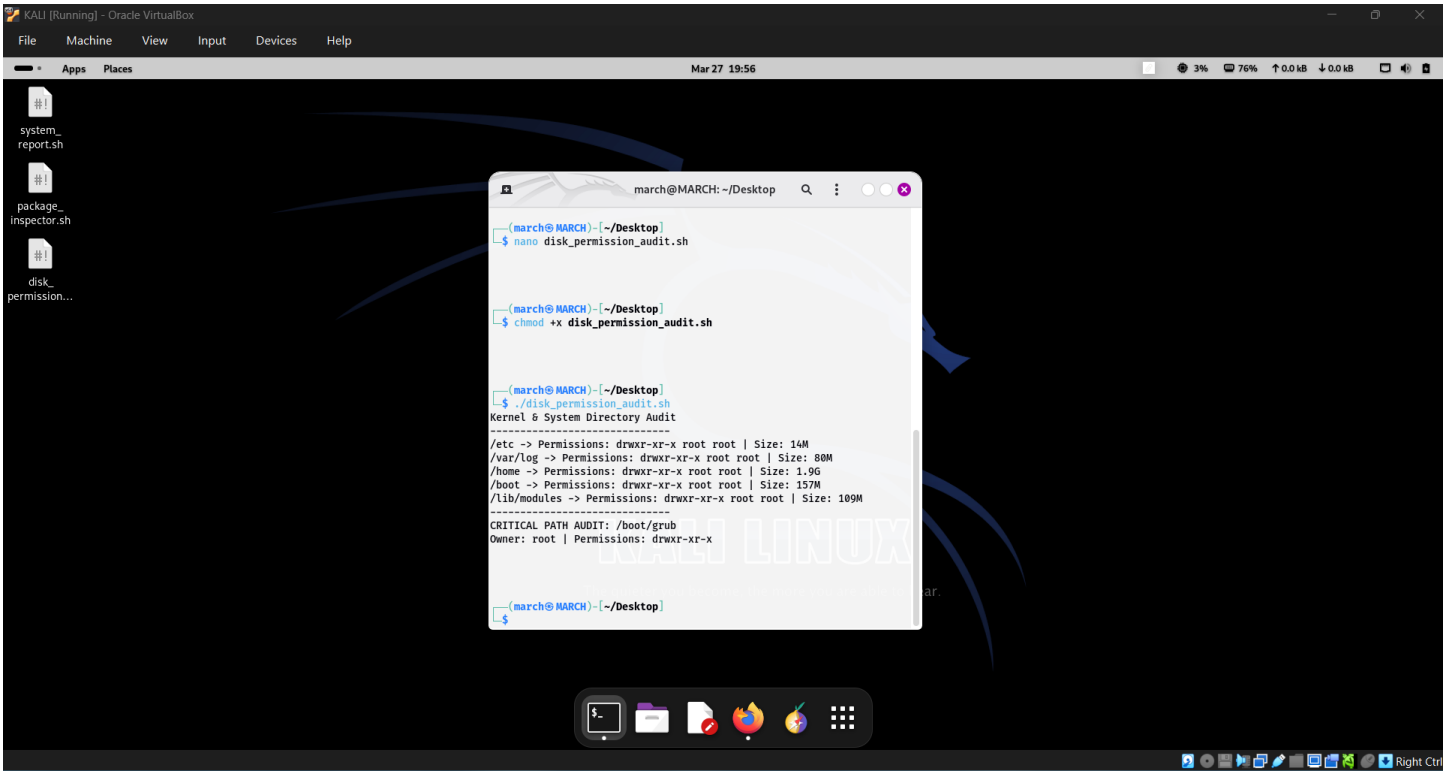
- The script first checks if the **current kernel package** (linux-image-$(uname -r)) is installed on the system by using dpkg -l and grep. If found, it prints a **verification message**.
- It then uses a **case statement** to check the package name and prints a corresponding **philosophical message** about the software.
    - If the package name starts with linux-image, it highlights the importance of system control and software freedom.
    - For packages like apache2 or httpd, it reflects on the role of web servers in shaping the open internet.
    - For database packages (mysql* or mariadb*), it comments on the significance of relational data management.
    - For other packages, it generally states their importance in the **FOSS ecosystem**.

- The use of **if-else** for binary checks (whether the package exists) and **case statements** for more complex conditions is an efficient way to process system information and provide meaningful feedback.

- Script 3: Security & Disk Audit

```
#!/bin/bash

# Script 3: Disk and Permission Auditor

# Loops through critical paths to audit security footprint.


DIRS=("/etc" "/var/log" "/home" "/boot" "/lib/modules")


echo "Kernel & System Directory Audit"

echo "-----------------------------"


for DIR in "${DIRS[@]}"; do

    if [ -d "$DIR" ]; then

        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')

        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        echo "$DIR -> Permissions: $PERMS | Size: $SIZE"

    fi

done


# Specific check for Kernel Configuration Path

GRUB_CONFIG="/boot/grub"

if [ -d "$GRUB_CONFIG" ]; then

    echo "-----------------------------"

    echo "CRITICAL PATH AUDIT: $GRUB_CONFIG"

    ls -ld "$GRUB_CONFIG" | awk '{print "Owner: "$3" | Permissions: "$1}'

fi
```

❖ *Output :-*



❖ *Logic Explanation :-*

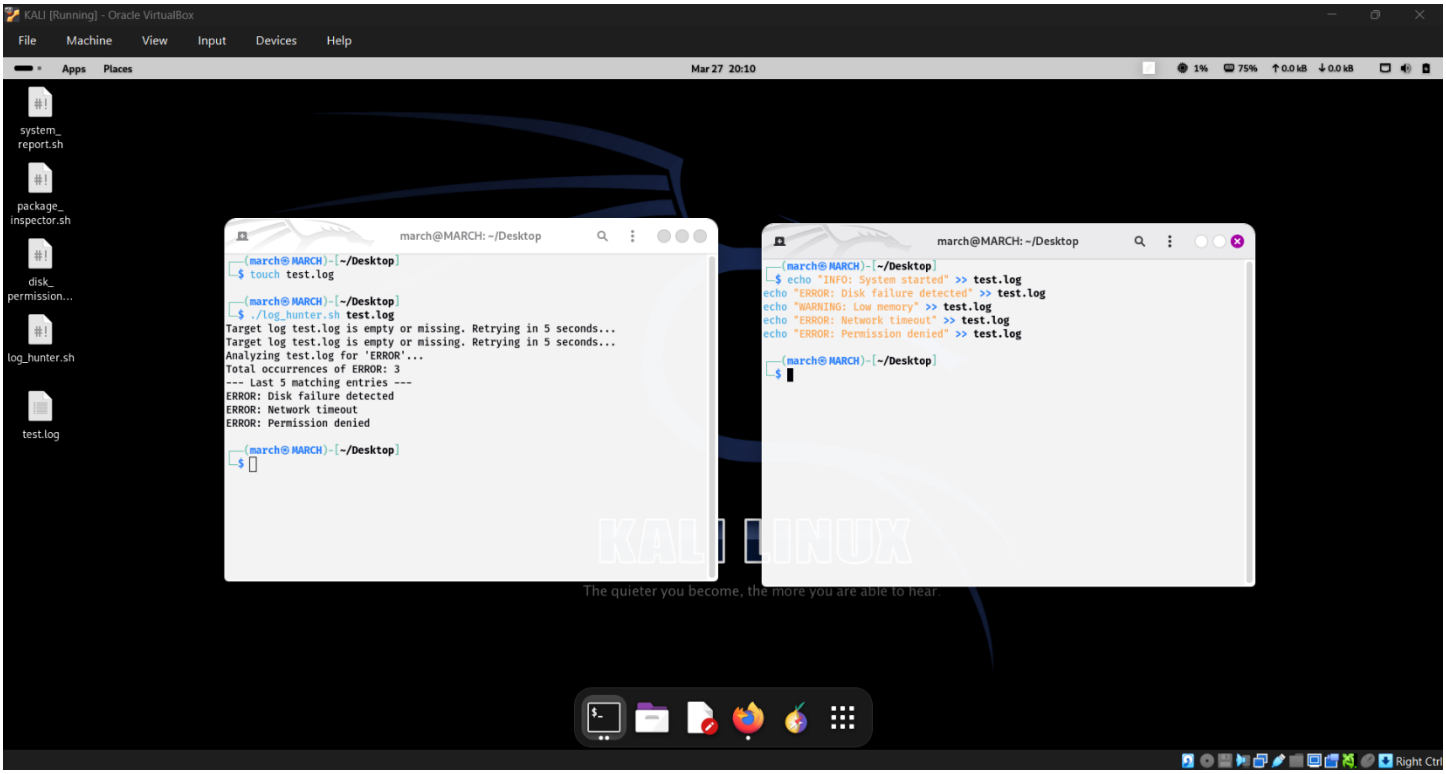
- The script uses a **for loop** to iterate over a predefined list of critical system directories (/etc, /var/log, /home, /boot, /lib/modules).
- For each directory, it checks if the directory exists using the -d condition. If it exists:
  - It retrieves and displays **permissions** using ls -ld.
  - It retrieves the **size** of the directory using du -sh.
- After auditing these directories, the script performs a **special audit** on the **/boot/grub** directory. This directory is crucial for bootloader configuration, so it is treated as a **critical path**.
- The script extracts the **owner** and **permissions** of /boot/grub using ls -ld and awk for column formatting.

- **awk** is used to extract and format specific columns from the output (e.g., owner, permissions).

- Script 4: Log Hunter (with Persistence)

```
#!/bin/bash

# Script 4: Log File Analyzer

# Implements a do-while style retry to handle empty or missing logs.


LOGFILE=$1

KEYWORD=${2:-"ERROR"}

COUNT=0


# Do-While Style Retry if file is empty or missing

while [ ! -s "$LOGFILE" ]; do

    echo "Target log $LOGFILE is empty or missing. Retrying in 5 seconds..."

    sleep 5

done


echo "Analyzing $LOGFILE for '$KEYWORD'..."


COUNT=$(grep -ic "$KEYWORD" "$LOGFILE")


echo "Total occurrences of $KEYWORD: $COUNT"

echo "--- Last 5 matching entries ---"

tail -n 100 "$LOGFILE" | grep -i "$KEYWORD" | tail -n 5
```

❖ *Output :-*



❖ *Logic Explanation :-*

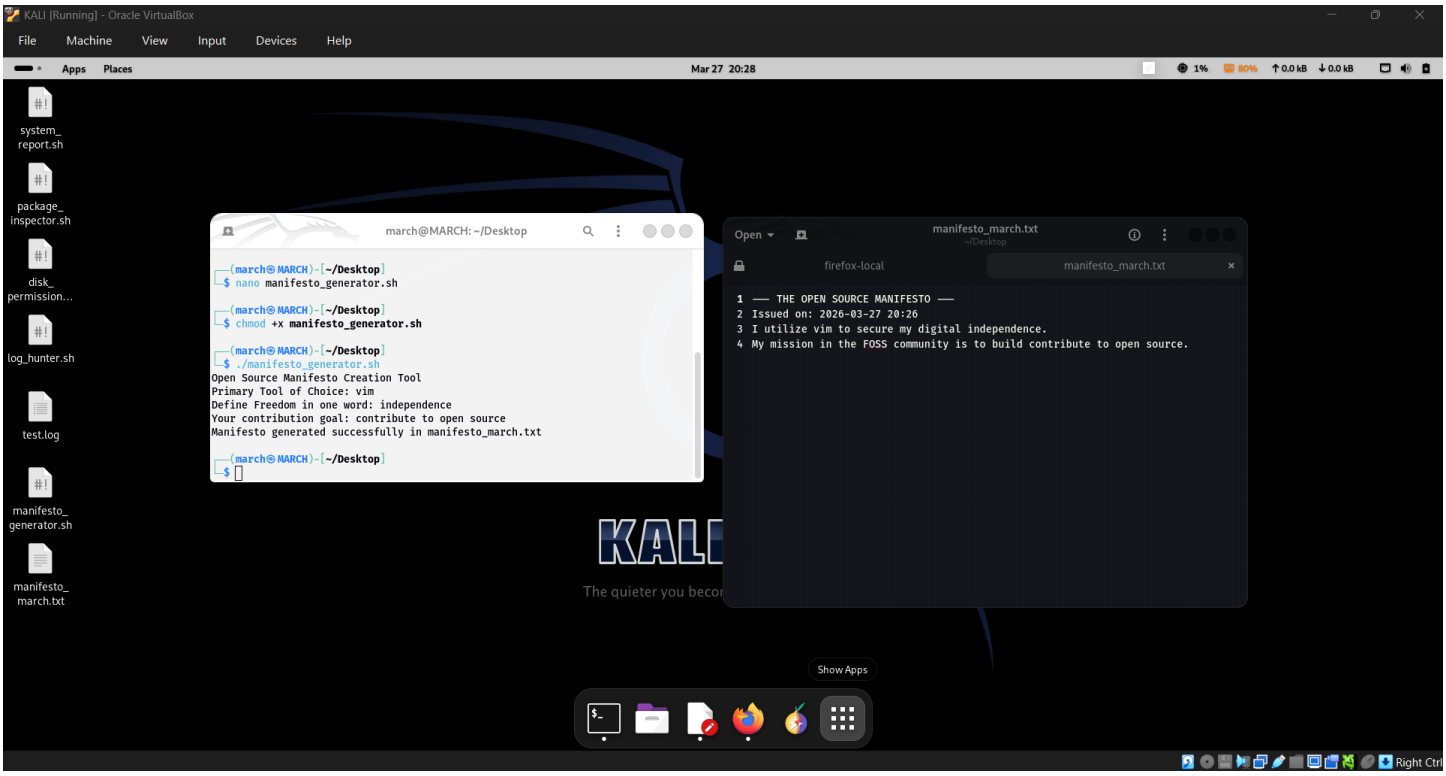
- The script takes a log file as input and optionally a keyword (default: "ERROR").
- It uses a **while loop with -s condition** to check whether the file exists and is not empty.
- If the file is missing or empty, it prints a retry message and waits for 5 seconds before checking again. This simulates a **do-while retry mechanism**.
- Once the file becomes available with content:
  - It counts occurrences of the keyword using grep -ic (case-insensitive count).
  - It extracts recent matches by:
    - Taking the last 100 lines using tail
    - Filtering keyword matches using grep
    - Displaying the last 5 entries using tail -n 5

- This approach ensures efficient **log monitoring and fault tolerance**.

- Script 5: Making Your Manifesto

```
#!/bin/bash

# Script 5: Manifesto Generator

# Generates a persistent record of software values.


echo "Open Source Manifesto Creation Tool"

read -p "Primary Tool of Choice: " TOOL

read -p "Define Freedom in one word: " FREEDOM

read -p "Your contribution goal: " BUILD


OUTPUT="manifesto_$(whoami).txt"

TIMESTAMP=$(date '+%Y-%m-%d %H:%M')


echo "--- THE OPEN SOURCE MANIFESTO ---" > "$OUTPUT"

echo "Issued on: $TIMESTAMP" >> "$OUTPUT"

echo "I utilize $TOOL to secure my digital $FREEDOM." >> "$OUTPUT"

echo "My mission in the FOSS community is to build $BUILD." >>
"$OUTPUT"


echo "Manifesto generated successfully in $OUTPUT"
```

❖ *Output :-*



❖ *Logic Explanation :-*

- The script starts by printing a message that it is creating an **Open Source Manifesto**.
- It then uses **read -p** to prompt the user for three inputs:
  - **Tool of Choice**: The primary software tool the user prefers.
  - **Definition of Freedom**: A single word that reflects the user's definition of freedom.
  - **Contribution Goal**: The user's goal in the **FOSS (Free and Open Source Software)** community.
- The inputs are stored in variables ($TOOL, $FREEDOM, $BUILD).
- The script then generates a **manifesto text file** with the name manifesto_username.txt, where username is the system's username (whoami).
- The **current timestamp** is added to the file using date.
- The content includes the user's manifesto, formatted in a clear, easy-to-read structure.
- Finally, a confirmation message is shown that the manifesto has been successfully generated.

7. Wrapping Up :-

The Linux Kernel is the ultimate success story for open-source. It survives because it has a great legal foundation and a community that cares about quality. For an architect, it's the best example of why open development is the way to go.

8. References :-

- **GNU Project:** "The Free Software Definition" (gnu.org)
- **Open Source Initiative:** "The Open Source Definition" (opensource.org)
- **Raymond, Eric S.:** *The Cathedral and the Bazaar* (1999)
- **Shotts, William:** *The Linux Command Line* (2019)